

PERFORMANCE EVALUATION AND DIMENSIONING OF NETWORKS AND SYSTEMS

METI

Lab Report [EN]

Authors:

José Oliveira (103771)

Tiago Videira (116069)

Afonso Mendes (116024)

jose.novais.oliveira@tecnico.ulisboa.pt

tiago.g.videira@tecnico.ulisboa.pt

afonsofmenDES@tecnico.ulisboa.pt

2026/2027 – 1st Semester

Contents

1	Introduction	2
2	A. Random number generators	3
2.1	Exercise 1	3
2.2	Exercise 2	3
3	B. Generation of random variables and stochastic processes	5
3.1	Exercise 3	5
3.2	Exercise 4	5
3.3	Exercise 5	5
4	C. Programming discrete event simulators	5
4.1	Exercise 6	5
4.2	Exercise 7	5
5	D. Estimation of probabilities	5
5.1	Exercise 8	5
6	E. Revisions of statistics	5
6.1	Exercise 9	5
6.2	Exercise 10	5
6.3	Exercise 11	5
7	F. Output data analysis	5
7.1	Exercise 12	5
7.2	Exercise 13	5
7.3	Exercise 14	5
8	Conclusion	5

1 Introduction

Text for the introduction here.

2 A. Random number generators

2.1 Exercise 1

Objective: Generate 1000 random numbers uniformly distributed in $[0, 1]$ using `runif()` and plot their histogram with `hist()`, to check informally whether the generator behaves as expected.

Approach: The following R script generates the sample and plots the histogram.

```
numbers <- runif(10000, min = 0, max = 1)
hist(numbers, main = "Histogram", xlab = "Value", ylab = "Samples",
      col = "lightblue", border = "black")
```

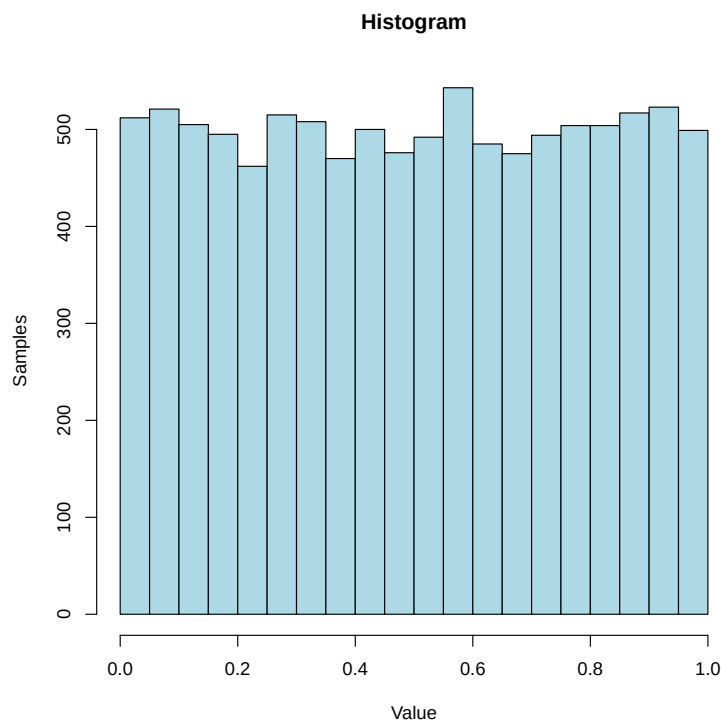


Figure 1: Histogram of 10000 samples generated with `runif()`.

Conclusion: As shown in Figure 1, the samples are approximately evenly distributed across all bins in $[0, 1]$, with no systematic bias toward any sub-interval. This is consistent with the expected behavior of a uniform distribution, confirming that `runif()` is generating numbers as intended.

2.2 Exercise 2

Objective: Verify, using `runif()`, that the sequence generated by a pseudo-random number generator is in fact deterministic, i.e., that it can be reproduced exactly.

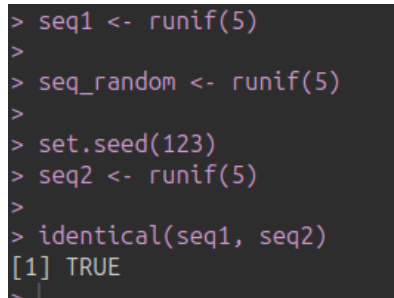
Approach: The `set.seed()` function fixes the internal state of R's random number generator. By setting the same seed before two separate calls to `runif()`, the two resulting sequences can be compared with `identical()` to check whether they match exactly. As a control, a third sequence is generated without resetting the seed, so it is expected to differ.

```
set.seed(123)
seq1 <- runif(5)

seq_random <- runif(5) # generated without resetting the seed

set.seed(123)
seq2 <- runif(5)

identical(seq1, seq2)
```



```
> seq1 <- runif(5)
>
> seq_random <- runif(5)
>
> set.seed(123)
> seq2 <- runif(5)
>
> identical(seq1, seq2)
[1] TRUE
>
```

Figure 2: R console output confirming that `seq1` and `seq2` are identical.

Conclusion: As shown in Figure 2, `identical(seq1, seq2)` returns `TRUE`, confirming that `seq1` and `seq2` are exactly the same sequence, even though they were generated by separate calls to `runif()`. This happens because `set.seed()` resets the internal state of the generator to the same fixed value before each call, so the pseudo-random number generator produces identical output whenever it starts from the same seed. This demonstrates that the sequence is not truly random but deterministic, fully determined by the seed. In contrast, `seq_random`, generated without resetting the seed, follows on from the generator's state after `seq1` and therefore differs from both.

3 B. Generation of random variables and stochastic processes

3.1 Exercise 3

3.2 Exercise 4

3.3 Exercise 5

4 C. Programming discrete event simulators

4.1 Exercise 6

4.2 Exercise 7

5 D. Estimation of probabilities

5.1 Exercise 8

6 E. Revisions of statistics

6.1 Exercise 9

6.2 Exercise 10

6.3 Exercise 11

7 F. Output data analysis

7.1 Exercise 12

7.2 Exercise 13

7.3 Exercise 14

8 Conclusion

Text for the conclusion here.